

Package placement map

Every package the curated stack builds, which layer it lives in, and why it lives there. Colours match the lanes-model deck. This is the companion to that deck: it carries the per-package detail the overview leaves out.

How a package's place is decided

Three questions, and they are independent. Collapsing them is what makes placement feel arbitrary; asking them separately makes it fall out.

- 1. **Is it compiler-dependent?** Must it be built with the surface's compiler to work with that surface's libraries?
- 2. **Is it lane-specific?** Does its build differ by execution model, or is one build right for Serial, MPI, and GPU alike?
- 3. **Is it automatically in the view, or explicitly loaded?** Does a user get it on their paths without asking, or do they load a module?

GROUP	COMPILER-SPECIFIC	LANE-SPECIFIC	AUTOMATICALLY IN VIEW
Foundation	in practice yes, built per compiler	no	yes
Core	not by nature (see below)	no	no
Compiler-common	yes	no	no
Payload lane	yes	yes	no

Foundation is defined by exposure behaviour, not by what the packages are. Something is Foundation when we want it silently reachable through the view and never loaded. That is why zlib and OpenBLAS feel alike, since neither has an MPI dimension, yet sit in different groups: one is linked without being chosen, the other is chosen.

On Core and compilers: Core packages are compiler-agnostic by nature, since a cmake is a cmake. We still build a Core per compiler, because one shared view cannot hold two compilers' `cmake/4.3.3` at the same path. "Not compiler-specific" describes the package, not where it is built.

Within a payload lane, one more question decides the build: an MPI-capable package is **dual-build**, `~mpi` in Serial and `+mpi` in MPI and GPU, under one clean module name that the loaded lane resolves.

The layers

Foundation

On your paths automatically through the compiler surface. Never a module.

PACKAGE	VERSIONS	WHY HERE
zlib	1.3.1	Linked without being chosen. Single version pinned by <code>require:</code> in the common scope, so every lane resolves the same one whether it is a root or a dependency.
xz	5.4.6	Same rule.
zstd	1.5.6	Same rule.

Core

Loads with `lcse/<Compiler>`, before any lane. Compiler-agnostic, or no MPI exists for it.

PACKAGE	VERSIONS	WHY HERE
cmake	4.3.3 · 4.2.3	Build tool, compiler-agnostic.
ninja · pkgconf · git	newest	Build tools. Unpinned: whatever the pinned recipe generation carries.
python	3.14.5 · 3.13.13	Compiler-agnostic building block. The two newest supported minor lines.
py-numpy	2.4.6 ×2	One build per python line. Both roots share a name and version, so the modules carry the line: <code>py-numpy/2.4.6-python3.14.5</code> . Qualified only because they collide.
miniforge3	26.1.1-3	Single by nature. It is the escape hatch for users who need their own package pairings.
gsl	2.8 · 2.7.1	No MPI implementation exists, and it behaves as a building block.
sqlite	3.53.1 · 3.51.2	Same rule.

Compiler-common

Built for this compiler, independent of the execution model. Its own group beside Core: loadable as soon as the compiler surface loads, before any lane, and the same module from Serial, MPI, or GPU.

PACKAGE	VERSIONS	WHY HERE
openblas	0.3.33 · 0.3.32	The single BLAS <i>and</i> LAPACK implementation: the recipe declares <code>provides("blas", "lapack")</code> . Compiler-dependent, so not Core; no MPI dimension, so not lane-specific. Every lane's users link BLAS, and the lanes are exclusive, so it has to be reachable from all of them.
gnuplot	6.0.0 · 5.4.10	User tool. Must be built with the surface's compiler to stay compatible with the libraries it links, which rules out Core; no execution-model dimension, which rules out a lane.

Serial lane

MPI-capable packages deliberately built without MPI. Each has a `+mpi` twin in the MPI and GPU lanes under the same name.

PACKAGE	VERSIONS	BUILT AS · PAIRING
hdf5	2.1.0 · 1.14.6	<code>~mpi +fortran +cxx +hl</code>
netcdf-c	4.10.0 · 4.9.3	<code>~mpi</code> , each pinned to one hdf5 (see chain pairing below)
netcdf-fortran	4.6.2 · 4.6.1	Rides its paired netcdf-c and hdf5
netcdf-cxx4	4.3.1	The recipe generation carries exactly one version; rides the newest chain only
fftw	3.3.11 · 3.3.10	<code>~mpi</code>
boost	1.90.0 · 1.89.0	<code>~mpi</code> . MPI-capable, so dual-build rather than common

MPI lane

Built against the system MPI, which is always an external and never built.

PACKAGE	VERSIONS	BUILT AS · PAIRING
hdf5	2.1.0 · 1.14.6	<code>+mpi +fortran +cxx +hl</code>
netcdf-c	4.10.0 · 4.9.3	<code>+mpi +parallel-netcdf</code> , same chain pairing as Serial
netcdf-fortran	4.6.2 · 4.6.1	Rides its paired chain
netcdf-cxx4	4.3.1	Newest chain only
fftw	3.3.11 · 3.3.10	<code>+mpi</code>
boost	1.90.0 · 1.89.0	<code>+mpi</code>

netlib-scalapack	2.2.3 · 2.2.2	Distributed dense linear algebra. MPI by nature: it is built on BLACS over MPI, so there is no serial form. Rides the lane's openblas for its BLAS and LAPACK
tau	2.35.1	<code>+mpi</code> . Profiler for MPI codes; no serial build offered. One version: a tool users run, not a library their code carries
dakota	6.24.0 · 6.23.0	<code>+mpi</code> . Heaviest build in the roster (large Boost-anchored tree)

GPU lane

The full MPI roster above, plus the GPU payload. Identical specs reuse the same installs by hash, so repeating the roster costs lockfile entries and modules, not rebuilds.

PACKAGE	VERSIONS	BUILT AS · PAIRING
the MPI roster	as above	Selected verbatim, so GPU stays a superset of the MPI lane's runtime surface
tau	2.35.1	<code>+mpi</code> plus the lane's GPU backend (<code>+rocm</code> or <code>+cuda</code>): its own build rather than the MPI lane's, which cannot see kernels. The variants are additive, so this one build profiles MPI codes, GPU codes, and MPI+GPU codes alike. No architecture flag, because tau traces the GPU runtime rather than compiling device code
kokkos	5.1.1 · 5.1.0	Built for the lane's GPU architecture, taken from the probe. It compiles device code, which is what proves the GPU toolchain works. New to the roster: it is here to prove that path rather than because users asked for it. See the note below on serving CPU lanes later

Never built: what the system provides

PACKAGE	PROVENANCE	WHY EXTERNAL
cray-mpich	Platform-backed	Fabric- and scheduler-integrated in ways a self-built MPI is not
openmpi	Site-external	The site's supported MPI on generic Linux
openssl · curl	Site-external	Track the OS security process, not ours
cray-libsci	system	Declared consumable on the Cray stacks. Carries BLAS, LAPACK, and ScaLAPACK, tuned for the platform, and the PrgEnv already places it on the path

Chain pairing

Each netcdf-c version is built against one named hdf5 rather than cross-multiplied, giving coherent stacks instead of untested mixtures. The same pairing holds in both the Serial and MPI forms.

CHAIN	HDF5	NETCDF-C	NETCDF-FORTRAN	NETCDF-CXX4
newest	2.1.0	4.10.0	4.6.2	4.3.1
predecessor	1.14.6	4.9.3	4.6.1	not built

hdf5 2.1.0 is the new major series and the recipes still mark 1.14.6 preferred. If 2.1.0 misbehaves on a real system, the 1.14.6 chain stands alone and 2.1.0 drops without touching anything else.

Settled recently

OpenBLAS is the one BLAS and LAPACK • decided 2026-07-14

netlib-lapack is dropped. OpenBLAS provides both APIs, so nothing loses a capability, and shipping both put LAPACK in the view twice: once inside `libopenblas.so`, once as `liblapack.so`. A user linking `-llapack` would silently get the unoptimized reference build, which is the ambiguity the Foundation pin rule prevents one layer up. Reference LAPACK returns only if a consumer needs it for numerical validation.

ScaLAPACK joins the MPI lane • decided 2026-07-14

Distributed dense linear algebra, built on BLACS over MPI, so it has no serial form and rides the lane's openblas. Two versions, 2.2.3 and 2.2.2.

One TAU, and the GPU lane builds its own • decided 2026-07-14

One version, because a profiler is a tool users run rather than a library their code carries. The GPU lane builds its own tau, with MPI and the GPU backend, rather than reusing the MPI lane's build, which cannot see kernels. The variants are additive, so one GPU-lane build covers MPI, GPU, and MPI+GPU codes; carrying both in that lane would collide on the name `tau/2.35.1` and fail loudly.

Open questions

MKL is in CSEinit today, not in this roster

The current flat stack offers MKL. Wheat is an Intel Cascade Lake system, where MKL is the vendor-tuned path. Dropping it silently would be a regression for those users; keeping it is a decision to make on purpose.

Kokkos could serve the CPU lanes too

Kokkos does not link MPI, so it is not an MPI-lane package in the way HDF5 is. It is a portability layer: one build carries a host backend and optionally a device backend, and a GPU build always has both. Today it sits in the GPU lane only, which means a Serial or MPI user has no Kokkos at all. A CPU build could sit in those lanes under the same name, the way Boost's two builds do, which would extend the dual-build rule from the MPI boundary to the GPU boundary.

Deferred for now, deliberately. Kokkos is in the roster to prove the GPU toolchain rather than to serve a request, and the reason to give CPU users a portability layer is performance, which this pass does not chase: everything builds at the portable baseline. Worth revisiting when target optimization turns on, or sooner if someone asks for Kokkos on CPU.

Vendor math libraries alongside OpenBLAS

On Blueback and Fran the PrgEnv already puts cray-libsci on the path, and we declare it consumable, so those users can reach two BLAS. LibSci and MKL are tuned per platform and both carry ScaLAPACK as well. The trade is real: OpenBLAS everywhere means users get the same library on every system and one set of instructions, while vendor libraries are faster on the machine they ship with. Neither is built from source; both are registered as externals, so this is a policy choice per system rather than a build cost.

AMD-tuned libraries, later

Three of the four pilot systems are AMD EPYC, where AOCL (BLIS plus libFLAME) is the vendor-tuned option. This is the same class of decision as CPU target optimization, which this pass deliberately defers: everything builds at the portable baseline first.

